

Reviewer Pack — Free Entropy of Disjointness Matrix Eigenvalues Bounds Commu...

Entry 736d55f9b51f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 01:45:20 UTC

Free Entropy of Disjointness Matrix Eigenvalues Bounds Communication Complexity

Entry ID: 736d55f9b51f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 01:45:20 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For any $n \times n$ communication matrix M of the DISJOINTNESS problem, the free entropy of the empirical spectral distribution of M is $\Omega(n)$.

Rationale (proposer's reasoning):

Free entropy in free probability quantifies the 'complexity' of non-commutative distributions. The eigenvalue distribution of M , which encodes structural information about the problem, may have a free entropy that scales with n , reflecting the inherent communication complexity of DISJOINTNESS.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e1cc8bfb9025f236

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            M[i][i] = 1
        return M

    def eigenvalues(matrix):
        n = len(matrix)
        if n == 1:
            return [matrix[0][0]]

        # Compute the characteristic polynomial using cofactor expansion
        det = 0
        for j in range(n):
            minor = [[matrix[i][k] for k in range(n) if k != j] for i in range(1, n)]
            det += (-1) ** j * matrix[0][j] * determinant(minor)
        return [det]

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        elif n == 2:
            return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]

        det = 0
        for j in range(n):
            minor = [[matrix[i][k] for k in range(n) if k != j] for i in range(1, n)]
            det += (-1) ** j * matrix[0][j] * determinant(minor)
        return det

    def logarithmic_potential(eigs):
        return sum(math.log(abs(eig)) for eig in eigs)
```

```

n = random.choice([5, 10, 15, 20, 30, 40])
M = generate_disjointness_matrix(n)
eigs = eigenvalues(M)
free_entropy = logarithmic_potential(eigs)

return {
    "metric_name": "free_entropy",
    "metric_value": free_entropy,
    "instances_tested": n,
    "conjecture_holds": free_entropy >= n,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_free_entropy = sum(r["metric_value"] for r in results) / len(results)
    std_free_entropy = math.sqrt(sum((r["metric_value"] - mean_free_entropy) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_free_entropy} std={std_free_entropy} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"free entropy < n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with stricter resource limits

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	49374
2	propose	ollama_remote	qwen3:8b	0	0	121815
3	propose	ollama_remote	qwen3:8b	0	0	104823
4	preregistration	ollama_remote	qwen3:8b	0	0	31621
5	novelty	ollama_remote	qwen3:8b	0	0	27343
6	novelty	ollama_remote	qwen3:8b	0	0	22867
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18790
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16548
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8604
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9837
11	judge	ollama_remote	qwen3:8b	0	0	19218

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 430840 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/736d55f9b51f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/736d55f9b51f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/736d55f9b51f.tar.gz* (if generated)